

Phase 12B Frontend Integration Specification

Platform Admin Authentication + Platform Control Plane Integration

Document: PHASE-12B-FRONTEND-INTEGRATION.md

Status: Implementation Specification

Target: Existing Next.js School ERP frontend

Backend: Existing Spring Boot Phase 12B Platform Control Plane

Frontend: Next.js App Router + React + TypeScript + Tailwind

Principle: Extend existing frontend; do not rebuild existing tenant frontend.

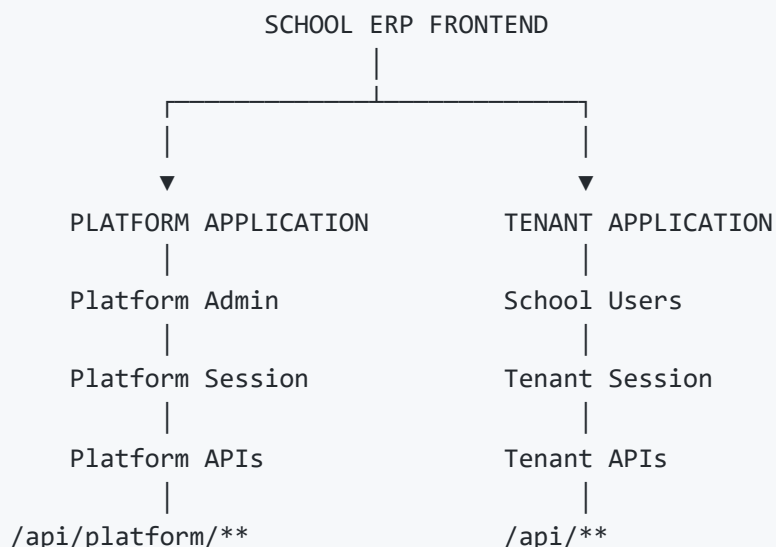
1. Purpose

Phase 12B backend introduced a new **Platform Control Plane**.

The existing frontend was primarily designed around the **Tenant/School application**.

The purpose of this frontend integration is to connect the existing frontend to the new backend platform APIs.

The result should be:



The backend architecture already establishes this separation.

2. Critical Requirement

Do NOT rewrite the existing frontend.

The existing frontend already contains:

- application shell
- dashboard
- tenant navigation
- admissions
- timetable
- communication
- reusable UI components
- design system
- authentication-related foundations/mock boundaries
- shared layouts
- responsive behavior

Phase 12B frontend work must therefore be an **incremental integration**.

Forbidden approach

```
Delete current frontend
      ↓
Create new frontend
      ↓
Reimplement existing modules
```

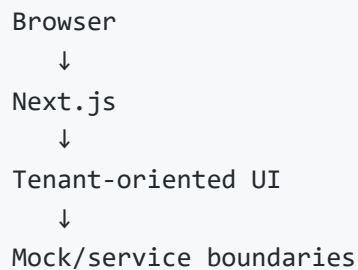
Required approach

```
Existing frontend
├── preserve Tenant application
├── preserve existing design system
├── preserve existing modules
└── add Platform application
    ├── Platform Login
    ├── Platform Session
    ├── Platform Shell
    ├── Platform Dashboard
    ├── Tenant Management
    ├── Plans
    └── Subscriptions
```

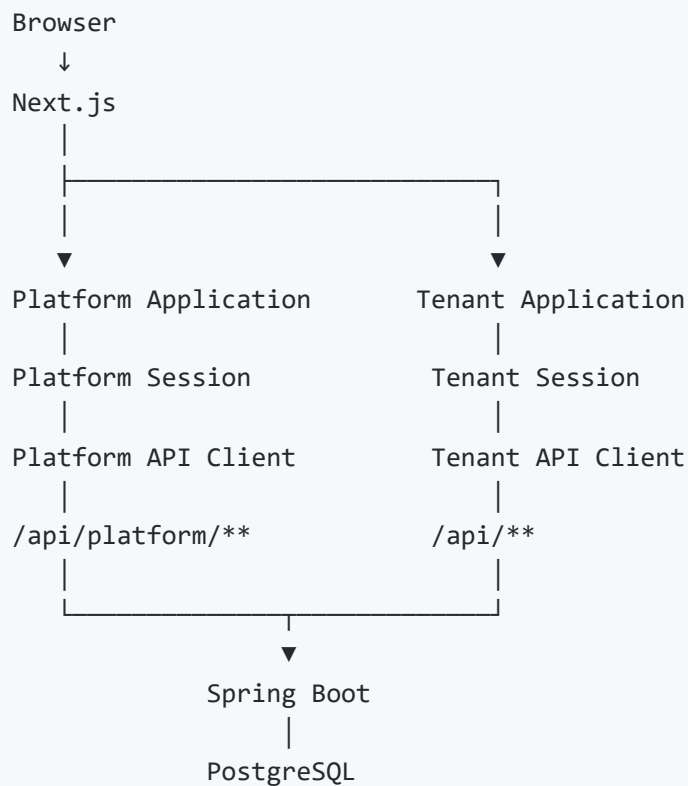
3. Current Architecture vs Target Architecture

Current frontend

The existing application effectively behaves like:



The frontend must evolve into:



The previous specification explicitly defines the desired platform/tenant API split.

4. Phase Scope

This frontend phase should be divided into:

```
12B-F1  Frontend API Foundation
12B-F2  Platform Authentication
12B-F3  Platform Session Management
12B-F4  Platform Route Protection
12B-F5  Platform Application Shell
12B-F6  Platform Dashboard
12B-F7  Tenant Management
12B-F8  Plans & Subscriptions
12B-F9  Platform User Management
12B-F10 Platform Audit
12B-F11 Tenant Authentication Integration
12B-F12 End-to-End Security Verification
```

Do **not** implement everything simultaneously.

The first milestone should be:

```
Platform Login
  ↓
Platform Authentication
  ↓
Platform Session
  ↓
Platform Dashboard
  ↓
Tenant List
```

This becomes the first complete frontend/backend vertical slice.

5. Fundamental Architecture Rule

The frontend must recognize two different security contexts.

Platform

```
Platform Admin
  ↓
Platform Authentication
  ↓
Platform Session
  ↓
Platform API
  ↓
/api/platform/**
```

Tenant

```
School User
  ↓
Tenant Authentication
  ↓
Tenant Session
  ↓
Tenant API
  ↓
/api/**
```

These must never be silently interchangeable.

6. Platform Admin Is NOT a Tenant User

This distinction must exist throughout the frontend.

Do not model:

```
type User = {
  role: "PLATFORM_ADMIN" | "TEACHER" | "STUDENT"
}
```

as the primary security model.

Instead conceptually:

```
Account
├── Platform authority
│   └── Platform roles
└── Tenant authority
    ├── Memberships
    └── Tenant roles
```

The backend intentionally separates these authorities.

Therefore the frontend should also separate:

```
PlatformSession
```

from:

The previous documentation explicitly identifies this as a critical distinction.

7. Existing Routes Must Remain Stable

Before implementation, Codex/developer must audit the current `src/app` tree.

The integration must identify:

```
existing routes
existing layouts
existing shells
existing route groups
existing providers
existing middleware
existing API abstractions
existing storage utilities
existing mock services
```

Then add platform routes without breaking them.

Target:

```
src/app/

existing tenant routes
    +
platform/
    login/
    dashboard/
    tenants/
    plans/
    subscriptions/
    users/
    audit/
    settings/
```

The exact route grouping may differ depending on the existing project structure.

Do not move existing routes merely for aesthetic consistency.

Route migration is allowed only if required and verified.

8. Platform Login

The first implementation target:

```
/platform/login
```

This is a dedicated Platform Admin login page.

It should contain:

```
School ERP  
Platform Administration
```

```
Email / Username  
Password
```

```
[ Sign In ]
```

The page must not contain:

- tenant selector
- school selector
- campus selector
- academic year selector
- student information
- tenant navigation
- tenant dashboard components

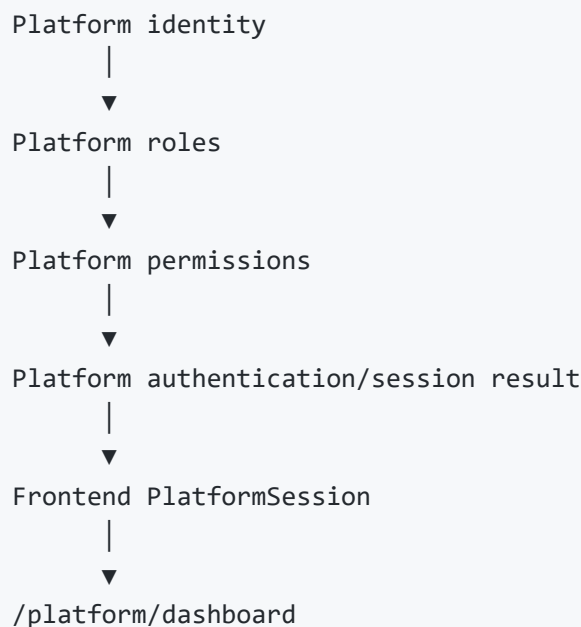
Why?

Because platform authentication establishes a **platform security context**, not a tenant membership context.

9. Login Flow

The expected flow is:

```
/platform/login  
  |  
  | credentials  
  ▼  
Platform Auth Service  
  |  
  ▼  
POST /api/platform/auth/login  
  |  
  ▼  
Backend authentication  
  |
```



The frontend must not decide:

```
"this user is admin"
```

based on a locally stored flag.

The backend response is authoritative.

The uploaded specification explicitly states that the backend must determine platform authority.

10. Backend API Contract Must Be Audited First

Before implementing the login page, inspect the actual Phase 12B backend implementation.

Do **not** invent request/response formats.

The implementation process must identify the actual:

```
POST /api/platform/auth/login
POST /api/platform/auth/refresh
POST /api/platform/auth/logout
GET  /api/platform/auth/me
```

or whatever exact routes exist in the implemented backend.

The frontend documentation should then record:

- HTTP Method
- Path
- Request DTO
- Response DTO
- HTTP status
- Authentication requirement
- Permission requirement
- Error contract
- Rate-limit behavior
- Session behavior

This is extremely important.

The earlier documentation used conceptual endpoint names; the **actual backend implementation must be treated as authoritative**.

11. API Contract Document

Create:

```
docs/frontend/PLATFORM-API-CONTRACT.md
```

Structure:

```
# Platform API Contract

## Authentication

### Login
### Refresh
### Logout
### Me

## Tenants

### List
### Get
### Create
### Suspend
### Reactivate
### Deactivate

## Plans

### List
### Get
### Create
### Update
```

```
## Subscriptions

### List
### Get
### Update

## Platform Users

### List
### Create
### Role Management

## Audit

### List
### Detail
```

Only document endpoints that actually exist.

12. Frontend API Architecture

Never do this inside components:

```
fetch("/api/platform/tenants")
```

Instead:

```
Component
  ↓
Feature API
  ↓
Platform API Client
  ↓
HTTP Client
  ↓
Backend
```

Recommended conceptual structure:

```
src/lib/api/
  http-client.ts
  platform-api.ts
  tenant-api.ts
```

and:

```
src/features/platform/  
  auth/  
  dashboard/  
  tenants/  
  plans/  
  subscriptions/  
  users/  
  audit/
```

The existing frontend architecture should be reused where it already provides equivalent abstractions.

Do not create duplicate HTTP infrastructure merely because the folder name differs.

13. HTTP Client

There should be one shared low-level HTTP mechanism.

Responsibilities:

```
base URL  
request headers  
credentials  
serialization  
deserialization  
timeout  
AbortSignal  
error normalization  
401 handling  
403 handling  
429 handling  
correlation ID
```

It should NOT contain:

```
tenant business logic  
platform business logic  
student logic  
admission logic
```

14. Platform API Boundary

Create a strongly typed platform API facade.

Conceptually:

```
platformApi.auth.login()
platformApi.auth.refresh()
platformApi.auth.logout()
platformApi.auth.me()

platformApi.tenants.list()
platformApi.tenants.get()
platformApi.tenants.create()
platformApi.tenants.suspend()
platformApi.tenants.reactivate()

platformApi.plans.list()
platformApi.plans.get()
platformApi.plans.create()
platformApi.plans.update()

platformApi.subscriptions.list()
platformApi.subscriptions.get()

platformApi.users.list()
platformApi.users.create()

platformApi.audit.list()
```

These names are conceptual. They must map to the **actual Phase 12B API**.

15. Tenant API Must Remain Separate

The existing tenant API infrastructure must not be replaced by the platform API.

Maintain:

```
platformApi
```

and:

```
tenantApi
```

independently.

```
platformApi
  ↓
/api/platform/**

tenantApi
```

↓
/api/**

This prevents an extremely dangerous class of bugs where a platform request accidentally receives tenant authentication or vice versa.

The uploaded specification explicitly requires that these API boundaries remain separate.

16. Session Model

Create explicit frontend session types.

Platform

`PlatformSession`

should represent:

```
authenticated
account identity
platform roles
platform permissions
session state
```

Tenant

`TenantSession`

should represent:

```
authenticated
account identity
membership ID
tenant ID
tenant roles
tenant permissions
session state
```

The tenant session has:

```
tenantId
membershipId
```

The platform session must not be converted into a fake tenant session.

17. Token Storage — Important Audit Requirement

This is an area where I would **not allow Codex to make an assumption**.

The frontend implementation must first inspect the actual Phase 12B authentication response and backend token/session architecture.

Determine:

```
Does backend return bearer access token?  
Does backend return refresh token?  
Are refresh tokens intended for HttpOnly cookies?  
Does backend support cookie authentication?  
Are tokens returned in JSON?  
What is token lifetime?  
What is refresh endpoint behavior?
```

Only after that should frontend token storage be implemented.

Never allow:

```
localStorage.accessToken  
localStorage.refreshToken
```

by default.

The previous documentation correctly warns against using ordinary local storage for authentication secrets.

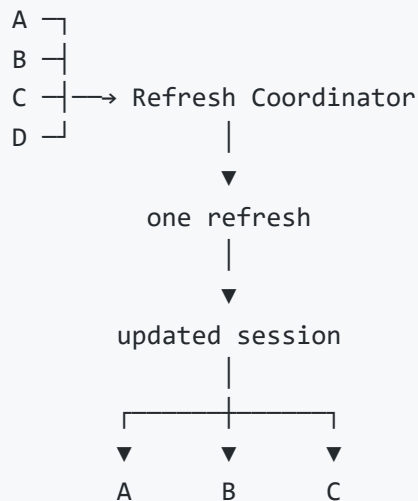
18. Refresh Concurrency

If bearer-token refresh is part of the actual backend contract, frontend refresh must be centralized.

Bad:

```
Request A → 401 → refresh  
Request B → 401 → refresh  
Request C → 401 → refresh  
Request D → 401 → refresh
```

Correct:



This is particularly important because the backend has already hardened refresh-token rotation and replay handling.

Frontend concurrency must not undermine that security model.

19. 401 and 403

Centralize the behavior.

401

```
Unauthenticated
  ↓
Session invalid
  ↓
redirect to correct login
```

Platform:

```
/platform/login
```

Tenant:

```
/login
```

403

```
Authenticated
but unauthorized
```

↓
Access Denied

Do not redirect a valid authenticated user to login simply because they lack a permission.

The previous specification explicitly defines this distinction.

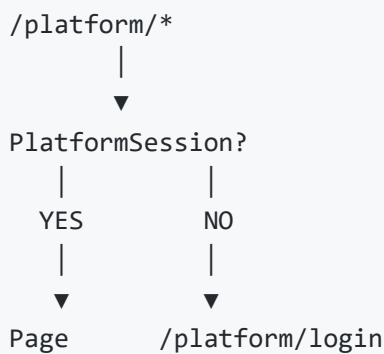
20. Platform Route Guard

Every:

```
/platform/**
```

route must require Platform authentication.

Conceptually:



But remember:

frontend route guards are UX/navigation protection, not the final security boundary.

The backend must still reject unauthorized API calls.

21. Platform Shell

Create:

```
PlatformAppShell
```

but reuse existing shared components.

For example:

Existing Design System



Tenant Shell Platform Shell

Do not create a second design system.

22. Platform Navigation

Initial navigation:

```
Dashboard
Tenants
Plans
Subscriptions
Platform Users
Audit
Settings
```

But navigation must be **permission-aware**.

For example:

```
platform.tenant.view
platform.plan.view
platform.subscription.view
platform.user.view
platform.audit.view
```

A user without:

```
platform.audit.view
```

should not see the Audit navigation item.

Again:

Frontend permission



UX

Backend permission



Security

23. Platform Dashboard

Do not invent statistics.

The dashboard should initially consume actual backend data.

If the backend does not yet expose a dashboard aggregate endpoint, do not make the frontend execute multiple inefficient queries merely to manufacture a dashboard.

Instead:

Option A:

Use existing aggregate endpoint

Option B:

Build dedicated platform dashboard endpoint

Option C:

Initially show operational cards based on existing supported APIs

Do not load hundreds/thousands of tenant records just to calculate:

Total tenants

Active tenants

Suspended tenants

in the browser.

The backend should own aggregate computation.

24. Tenant Management

Implement:

/platform/tenants

using the actual backend tenant-management API.

Requirements:

```
pagination
sorting
search
status filtering
loading state
empty state
error state
permission checks
```

Do not fetch every tenant and filter locally.

25. Tenant Creation

Create:

```
/platform/tenants/new
```

or a modal/drawer depending on the existing UI architecture.

The UI should reflect the backend provisioning model:

```
Tenant information
  ↓
Initial admin
  ↓
Plan
  ↓
Subscription
  ↓
Provision
```

The backend provisioning flow is transactional and idempotent.

26. Idempotency

For creation/mutation requests where the backend supports idempotency:

```
Idempotency-Key: UUID
```

must be generated once per logical user operation.

Example:

```
User clicks Create
      ↓
Generate key
      ↓
Request
      ↓
Network timeout
      ↓
Retry
      ↓
Same key
```

Never generate a new key for an automatic retry of the same operation.

27. Plans and Subscriptions

Frontend must reflect the backend model:

```
Plan
  ↓
PlanVersion
  ↓
Subscription
  ↓
SubscriptionHistory
```

Do not design the UI around:

```
Tenant
  ↓
planId
```

because that would contradict the backend domain model.

The backend deliberately uses versioned plans so historical subscriptions are not silently mutated.

28. Platform Users

Platform users are not tenant employees.

The UI should therefore have:

```
/platform/users
```

rather than placing them under:

```
/teachers  
/employees
```

Platform user functionality should represent:

```
Platform identity  
Platform roles  
Platform permissions  
Platform session/security state
```

29. Platform Audit

Create:

```
/platform/audit
```

The UI should support whatever the backend actually exposes, likely:

```
timestamp  
actor  
action  
target  
result  
correlation/trace information
```

Do not expose:

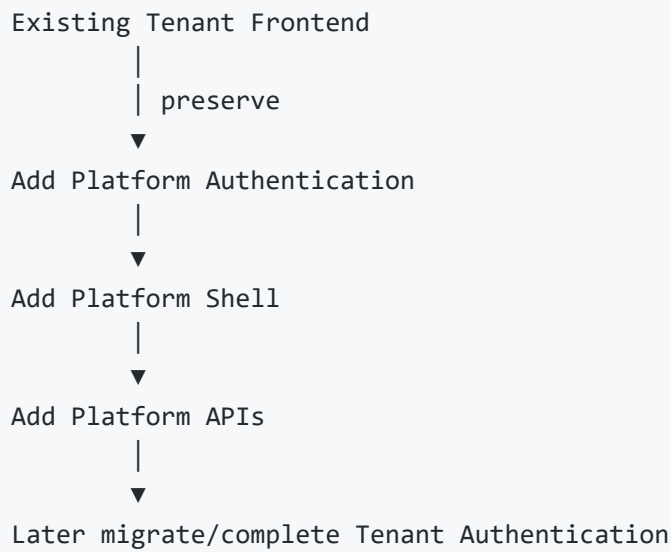
```
passwords  
JWTs  
refresh tokens  
raw lifecycle tokens  
sensitive secrets
```

The backend already treats platform audit as an important security boundary.

30. Existing Tenant Login

Do **not** replace the existing tenant application login while implementing Platform Admin login unless the backend contract requires it.

The preferred incremental sequence is:



This minimizes regression risk.

31. Existing Mock Services

Your existing frontend contains mock-oriented boundaries.

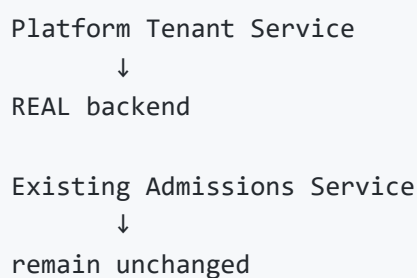
Do not delete them blindly.

First classify each:

MOCK ONLY
REAL API READY
SHARED INFRASTRUCTURE
LEGACY
UNUSED

Then migrate only the Platform features that are now backed by real APIs.

For example:



until its own integration phase.

32. Existing appStorage

Your frontend already has an `appStorage` abstraction.

Do not automatically reuse it for security credentials.

Audit its current responsibilities.

If it is used for:

```
theme
sidebar state
UI preferences
filters
```

that is different from:

```
authentication credentials
```

Authentication storage must have its own explicit security model.

33. Existing App Shell

You already have an existing:

```
AppShell
```

with tenant/school/campus/year selectors and tenant navigation.

Do not add:

```
if (isPlatformAdmin) {
  ...
}
```

everywhere inside that shell.

Instead:

```
Shared design primitives
|
```

```
└─ TenantAppShell
└─ PlatformAppShell
```

This keeps the two contexts understandable.

34. Avoid Security Context Contamination

A major frontend threat is:

```
Platform Login
  ↓
Platform Session
  ↓
Tenant AppShell
```

or:

```
Tenant Login
  ↓
Tenant Session
  ↓
Platform API
```

Prevent this structurally.

For example:

```
Platform pages
  import platform session/API modules

Tenant pages
  import tenant session/API modules
```

Do not expose one generic:

```
useAuth()
```

that silently returns whichever context happens to be active.

Prefer explicit concepts such as:

```
usePlatformSession()
useTenantSession()
```

where appropriate.

35. Error Contract

The frontend should normalize backend errors.

Target:

```
Spring Problem Details
  ↓
HTTP client
  ↓
Frontend ApiError
  ↓
UI
```

The UI should distinguish:

```
400 Validation
401 Authentication
403 Authorization
404 Not Found
409 Conflict
422 Business Validation
429 Rate Limit
500 Server Error
```

The exact status/code mapping must follow the actual backend contract.

36. Rate Limiting

The backend already has rate-limit behavior.

Therefore login UI must handle:

```
429 Too Many Requests
```

properly.

Example:

```
Too many login attempts.

Please wait before trying again.
```

If backend sends:

```
Retry-After
```

the frontend should respect it.

Do not blindly retry authentication requests.

37. Loading States

Platform login:

```
idle  
→ submitting  
→ success
```

During submission:

```
Sign In
```

becomes:

```
Signing in...
```

and duplicate submission must be prevented.

38. Authentication Error UX

Do not leak backend security details.

Avoid showing:

```
Account exists but platform role missing
```

unless that is intentionally part of the public error contract.

Prefer a safe message such as:

```
Unable to sign in with these credentials.
```

while logging diagnostic information only through approved observability mechanisms.

39. Redirect Rules

Platform:

```
/platform/login
```

If already authenticated:

```
/platform/dashboard
```

Platform protected route:

```
/platform/tenants
```

if unauthenticated:

```
/platform/login?returnTo=/platform/tenants
```

However, validate/sanitize `returnTo` .

Never allow open redirects such as:

```
returnTo=https://malicious-site.example
```

40. Browser Security

The frontend integration must audit:

```
XSS  
CSRF  
CORS  
cookie attributes  
token exposure  
console logging  
error logging  
open redirects  
URL credential leakage  
source maps
```

Especially:

Do not put tokens into URLs.
Do not put passwords into query parameters.
Do not log Authorization headers.

41. API Configuration

Centralize:

```
API base URL
environment
request timeout
feature flags
```

For example:

```
NEXT_PUBLIC_API_BASE_URL
```

But don't hardcode environment-specific URLs in components.

The frontend should work against:

```
development
test
staging
production
```

without source-code changes.

42. TypeScript Contract

No:

```
any
```

for platform API responses.

Create:

```
PlatformLoginRequest
PlatformLoginResponse
PlatformSession
PlatformUser
```

```
PlatformRole
PlatformPermission
TenantSummary
TenantDetail
CreateTenantRequest
Plan
PlanVersion
Subscription
PlatformAuditEvent
ApiError
PageResponse<T>
```

but only where supported by the actual backend.

43. Do Not Duplicate Business Logic

Frontend should not implement:

```
tenant lifecycle rules
subscription entitlement calculation
platform authorization
security-version validation
membership authorization
```

Instead:

```
Frontend
  ↓
request
  ↓
Backend
  ↓
authoritative business/security decision
```

Frontend may display the result.

44. Testing Strategy

The integration needs multiple levels.

Unit tests

Test:

```
HTTP client
API mappers
error normalization
permission helpers
session state transitions
```

Component tests

Test:

```
login form
permission gate
platform navigation
tenant table
tenant creation form
```

Integration tests

Test:

```
login → session → dashboard
dashboard → tenant API
logout → protected route
401 → refresh/login
403 → access denied
```

E2E

Test the real system:

```
Browser
↓
Next.js
↓
Spring Boot
↓
PostgreSQL
```

45. Critical Security E2E Tests

These are mandatory.

Test 1

```
Platform Admin
  ↓
Platform Login
  ↓
SUCCESS
```

Test 2

```
Invalid platform credentials
  ↓
LOGIN FAILURE
```

Test 3

```
Tenant user
  ↓
/platform/dashboard
  ↓
DENIED
```

Test 4

```
Tenant JWT
  ↓
/api/platform/...
  ↓
401/403
```

Test 5

```
Platform JWT
  ↓
tenant API
  ↓
401/403
```

The backend already verifies cross-plane JWT rejection; frontend E2E should ensure the browser experience respects that same boundary.

46. Definition of Done for First Milestone

Do not call the first milestone complete until this works:

```
Browser
↓
/platform/login
↓
enter credentials
↓
real POST /api/platform/auth/login
↓
real backend authentication
↓
real platform session
↓
/platform/dashboard
↓
real platform API request
↓
real backend data
```

And:

```
Logout
↓
session invalidated
↓
/platform/dashboard
↓
redirect to /platform/login
```

47. Definition of Done for Full Frontend Integration

The complete Phase 12B frontend integration is complete only when:

Platform Login	✓
Platform Session	✓
Platform Logout	✓
Platform Refresh	✓
Platform Route Guard	✓
Platform Shell	✓
Platform Dashboard	✓
Tenant Management	✓
Tenant Provisioning	✓
Plans	✓
Subscriptions	✓
Platform Users	✓
Platform Audit	✓
Permission-aware UI	✓
API error handling	✓

Rate-limit handling	✓
Loading/empty/error states	✓
Platform/Tenant isolation	✓
E2E verification	✓
Production frontend audit	✓

48. Documentation Deliverables

I recommend adding these to your existing project documentation:

```
docs/
|
├── frontend/
|   |
|   ├── FRONTEND-ARCHITECTURE.md
|   ├── PLATFORM-AUTHENTICATION.md
|   ├── PLATFORM-API-CONTRACT.md
|   ├── PLATFORM-SESSION.md
|   ├── PLATFORM-ROUTING.md
|   ├── PLATFORM-UI.md
|   ├── PLATFORM-PERMISSIONS.md
|   └── FRONTEND-SECURITY.md
|
├── platform/
|   ├── PLATFORM-CONTROL-PLANE.md
|   ├── TENANT-LIFECYCLE.md
|   ├── PROVISIONING.md
|   ├── SUBSCRIPTIONS.md
|   └── AUDIT.md
|
└── phases/
    └── phase-12b-frontend-integration.md
```

Don't duplicate information unnecessarily. Cross-reference the authoritative backend documents.

49. Recommended Actual Work Order

For **your existing project**, I would implement it exactly in this order:

PHASE 12B FRONTEND INTEGRATION



01. Audit existing frontend

|

- └─ routes
- └─ AppShell
- └─ providers
- └─ storage
- └─ API clients
- └─ mock services
- └─ auth code



02. Audit actual Phase 12B APIs

- |
- └─ login
- └─ refresh
- └─ logout
- └─ me
- └─ tenants
- └─ plans
- └─ subscriptions
- └─ users
- └─ audit



03. Build common HTTP infrastructure



04. Build Platform API client



05. Build Platform session model



06. Implement /platform/login



07. Connect REAL login API



08. Implement session restoration



09. Implement logout



10. Implement 401/403 handling



11. Implement Platform route protection



12. Build PlatformAppShell



13. Build Platform Dashboard



14. Connect Tenant Management



15. Connect Tenant Provisioning



16. Connect Plans



17. Connect Subscriptions



18. Connect Platform Users



19. Connect Platform Audit



20. E2E security verification

50. Most Important: What Codex Should Do First

Before Codex changes anything, it should perform an **audit-only pass**.

It should answer:

1. What routes already exist?
2. What is the current AppShell architecture?
3. What authentication code already exists?

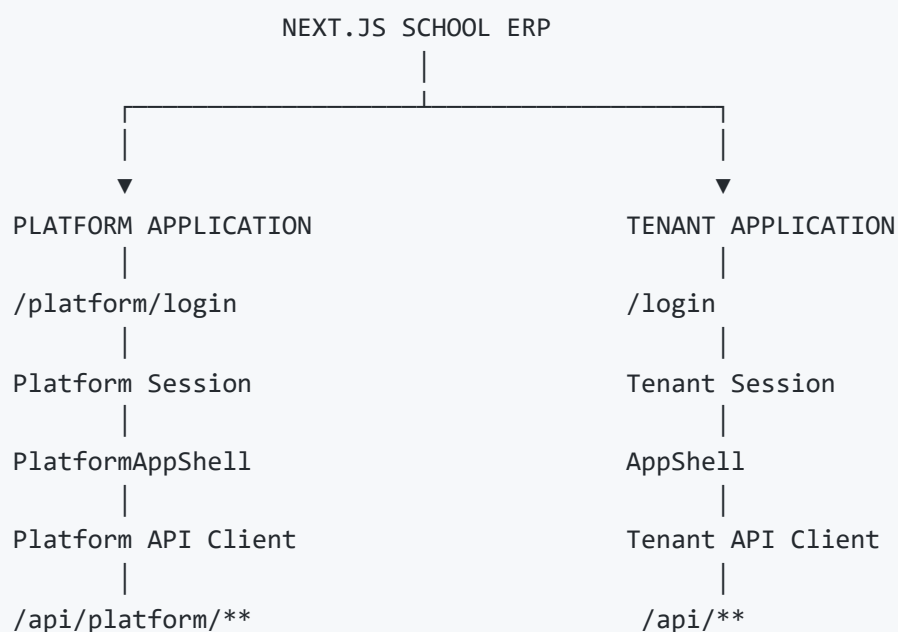
4. What is currently mock-only?
5. What API client already exists?
6. How is appStorage currently used?
7. What providers/context exist?
8. What middleware exists?
9. What backend Phase 12B authentication endpoints actually exist?
10. What are the exact login request/response DTOs?
11. How are platform tokens/session credentials actually returned?
12. What are the exact platform permissions?
13. What are the exact platform API routes?
14. Which frontend components can be reused?
15. Which existing code must remain untouched?
16. What changes are required to integrate Platform Admin?

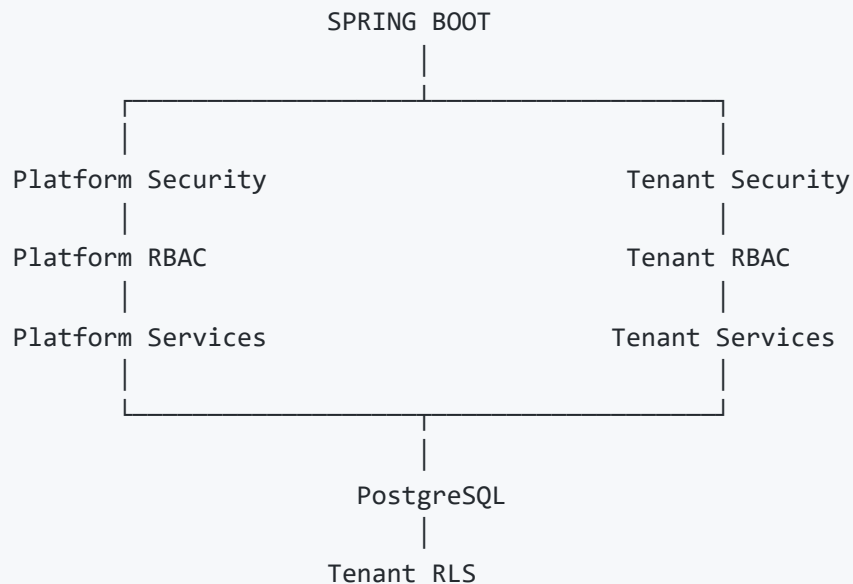
Only after this audit should implementation begin.

That is the key difference between a safe integration and having Codex blindly create a second frontend architecture.

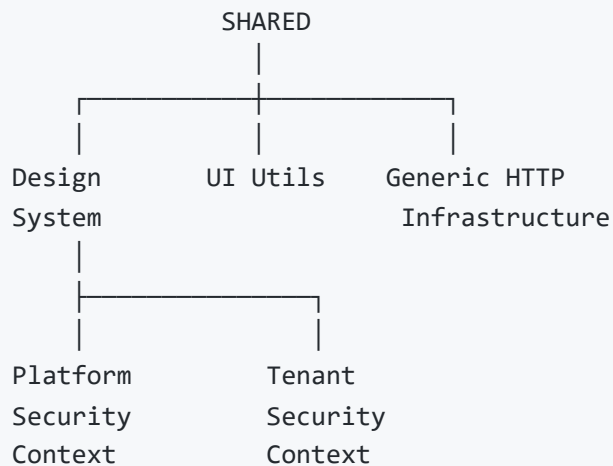
51. Final Architecture

Your final frontend should look approximately like this:





And importantly:



52. The First Concrete Deliverable

I would make the **first implementation milestone** extremely narrow:

Milestone 12B-F1

```
Existing frontend
↓
Add /platform/login
↓
Real Phase 12B platform login API
↓
Platform session
↓
Platform route guard
↓
```

```
PlatformAppShell
  ↓
/platform/dashboard
  ↓
Real "me" API
```

Then:

Milestone 12B-F2

```
/platform/tenants
  ↓
real tenant list API
  ↓
pagination
  ↓
permissions
  ↓
tenant detail
```

Then:

Milestone 12B-F3

```
Tenant provisioning
Plans
Subscriptions
```

Then:

```
Platform Users
Audit
Settings
```

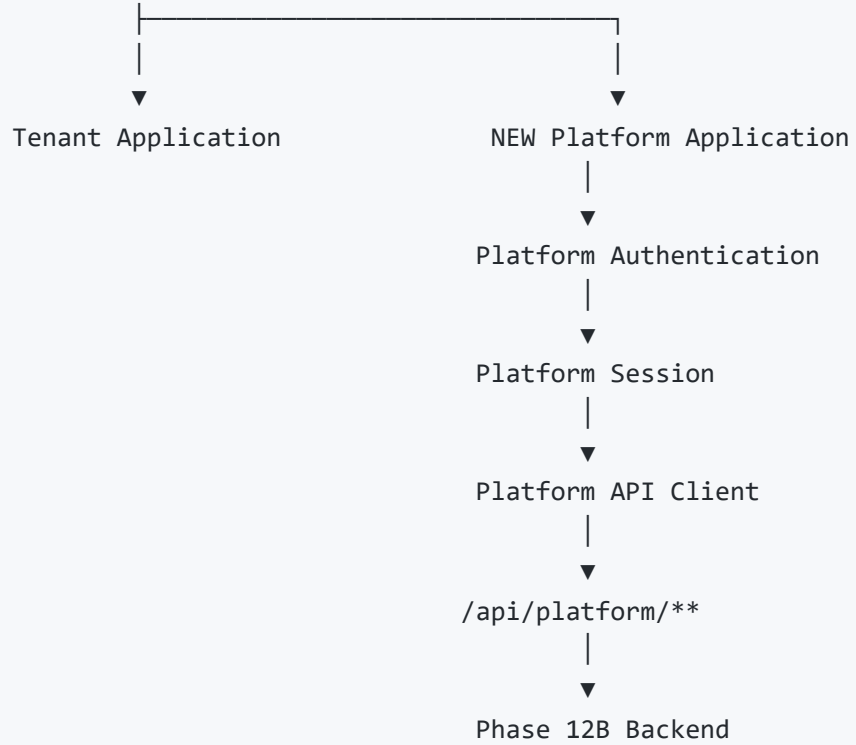
This is much safer than asking Codex to “implement the whole Phase 12B frontend.”

The key principle

Your Phase 12B backend is already the **Platform Control Plane**. The frontend work should therefore be an **integration layer**, not another redesign.

The architecture should remain:

```
EXISTING TENANT FRONTEND
|
| unchanged as much as possible
|
```



That is the implementation boundary I would freeze before giving Codex the coding task.